

## **Report Progetto S2-L5: Esercizio di Osservazione** **Critica su un Codice Python**

### **Obbiettivo:**

Dato il seguente codice si richiede allo studente di:

1. Capire cosa fa il programma senza eseguirlo.
2. Individuare nel codice sorgente le casistiche non standard che il programma non gestisce (per esempio, comportamenti potenziali che non sono stati contemplati).
3. Individuare eventuali errori di sintassi / logici.
4. Proporre una soluzione per ognuno di essi.

```
1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
23
```

## **Descrizione Operato:**

### **1. Capire cosa fa il programma senza eseguirlo:**

Il codice fornito in Python è di un programma che simula un assistente virtuale a cui l'utente può chiedere quale sia la data corrente, quale sia l'ora attuale e come si chiama l'assistente, o eventualmente terminare il programma scrivendo "esci" in input.

### **2. Individuare nel codice sorgente le casistiche non standard che il programma non gestisce:**

Per prima cosa si nota che nel codice non è presente da nessuna parte la condizione

`".lower"`, questa fa sì che il programma riesca a leggere gli input dell'utente, trasformando tutti i caratteri in minuscolo, anche utilizzasse delle lettere maiuscole. Mentre in questo caso, dato che non è stata inserita, se l'utente inserisce un input con delle lettere maiuscole, il programma non riesce a leggerlo correttamente.

Inoltre, sempre secondo la stessa logica, essendo che chi ha scritto questo codice ha utilizzato il simbolo `"=="` (righe 5, 12, 15 e 18), prevede come input leggibile solo quelle stringhe ESATTAMENTE come sono scritte. Quindi, ad esempio, se l'utente inserisse in input uno spazio di troppo o di meno, un segno di punteggiatura in più o in meno, un errore di battitura o di ortografia, o anche semplicemente un input vuoto, rispetto alle opzioni previste dal codice, il programma non riuscirebbe a leggere l'input correttamente e restituirebbe un errore.

### **3. Individuare eventuali errori di sintassi / logici:**

- I. Tutta la parte di codice dove si va a definire la funzione `"def assistente_virtuale(comando)"` che va da riga 11 a riga 22 dovrebbe essere scritta prima del ciclo `"while"` a riga 3 (che richiama la funzione stessa), questo è un errore di sintassi;
- II. A riga 3 dovrebbero esserci i `":"` dopo `"while True"`, per poter avviare correttamente il ciclo, questo è un errore di sintassi;
- III. A riga 7 il `"break"` non è indentato correttamente sotto `"if"`, di conseguenza il ciclo creato dal `"while"` verrebbe sempre interrotto dopo la prima iterazione, questo è un errore logico;
- IV. A riga 13 il comando della funzione che dovrebbe restituire la data corrente all'utente qualora richiesta, ovvero `"datetime.datetime.today()"` è scritto in modo sbagliato poiché manca un `."`, la scrittura corretta dovrebbe essere `"datetime.date.today()"`, questo è un errore di sintassi;
- V. A riga 16 il comando della funzione che dovrebbe restituire l'ora attuale all'utente qualora richiesta, ovvero `"datetime.datetime.now().time()"` è scritto in modo sbagliato, la scrittura corretta dovrebbe essere `"datetime.datetime.now()"`, questo è un errore di sintassi;

#### 4. Come migliorerei questo codice?

Innanzitutto, aggiustati tutti gli errori precedentemente illustrati nel rispettivo modo, se dovessi migliorare questo codice ecco cosa farei:

- I. Innanzitutto, dopo `import datetime` all'inizio del codice, inserirei un `print` per fornire all'utente una sorta di menù introduttivo, con un messaggio di benvenuto, l'elenco di tutto quello che potrebbe chiedere a questo assistente virtuale e l'input da scrivere per terminare il programma, del tipo:

```
1
2
3  print("""Benvenuto!
4  Sono il tuo assistente virtuale personale.
5  Questo è quello che puoi chiedermi:
6  Qual è la data di oggi?
7  Che ore sono?
8  Come ti chiami?
9  Termina il programma scrivendo 'esci'. """)
10
```

- II. Successivamente, come illustrato in precedenza, aggiungerei un `.lower()` all'input dell'utente a riga 4, così che il programma possa convertire tutta la stringa inserita in minuscolo. In questo modo sia che l'utente scriva "CIAO" che "Ciao" che "ciAO" ecc, il programma potrebbe leggere l'input correttamente.

La riga di codice risulterebbe così:

```
comando_utente = input("Cosa vuoi sapere? ").lower()
```

- III. Infine, a livello generale, migliorerei l'associazione degli input dell'utente alla relativa risposta, includendo le eccezioni precedentemente illustrate (errori di ortografia, battitura, input vuoti o non previsti ecc...) con diversi `try/except`, del tipo:

```
6  try:
7      # input che potrebbe dare un errore
8  except:
9      # cosa deve fare l'utente se si verifica un errore
```